

有界 MPMC 队列

从那个迫使一切设计决策的唯一冲突出发,推导 Vyukov 的无锁多生产者多消费者队列

Robert (Bo) Hou · Robert's Technical Notes · 2026 年 6 月

摘要。有界多生产者多消费者(MPMC)队列是无锁队列设计的集大成者。本报告不把 Vyukov 算法当作一个供人背诵的既成品抛出;它把算法重建为一连串被逼出来的步子,每一步都由上一步引发。首先确立那个唯一的根因:为了获得并行,占位(认领队列位置)必须与写入数据解耦,这意味着 tail 计数器在载荷写入之前就推进了——tail “撒了谎”。此后每一个特性,都是为了修补这个谎言的后果而存在。我们说明:为什么一对全局 head/tail 无法告诉消费者某个具体槽位是否就绪;为什么每槽一个版本号 token 是最小的修补;为什么这个 token 必须用一个单调值同时编码“第几圈”与“读/写相位”;为什么生产者不能对槽 token 用 CAS(CAS 会污染它必须保住的那张票)而只能 load-spin;以及为什么 acquire 必须落在循环退出那次 load、而非循环前那次。最终代码经多线程压测验证。全程的重点是那股使每个决策不可避免的张力——因为理解队列为何长成这样,远比复现它长什么样更有价值。

1. 根因:占位与写入必须解耦

在单生产者环形缓冲区里,生产者按程序顺序做两件事:先写槽,再推进 tail。因为是同一个线程顺序地做这两件事,tail 的推进必然跟在写入完成之后。于是 tail 带着一个诚实的含义——“在我之下的一切都已写完”——观察到 tail 的消费者可以信任它。

MPMC 击碎了这一点。当有多个生产者时,每个生产者将要写入的位置,必须由单条原子指令(对 tail 的 CAS 或 fetch-add)原子且迅速地分配出去。这个分配不能等数据写完——若要等,写入就串行化了,而我们正是为并行才造 MPMC 的。于是顺序被反转:生产者先认领一张票(推进 tail),之后才写它的载荷。

tail 的谎言(唯一的根因)。因为认领票与写入载荷被解耦,tail 在对应数据被写入之前就推进了。tail 不再意味着“在我之下的数据已就绪”;它只意味着“这些位置已被发放”。信任 tail 的消费者,可能去读一个其生产者已认领了票、却尚未写完——甚至尚未开始写——的槽。MPMC 队列其余的每一个特性,都是为补偿这唯一的谎言而存在。

这是必须首先说清的一句话。一上来就讲“每个槽带一个版本号”的讲法,是在病因之前先教解药,读者于是把 token 当成咒语背下来。token 不是出发点的公理;它是“把认领与写入解耦”这个决定被逼出的后果。从别处开篇,设计就显得凭空而来。

2. 复用槽上的三类冲突

剥掉一切机制,问:当两个线程触碰循环数组的同一个物理槽 `buf[i]` 时,能出什么错?因为槽 `i` 每 `N` 张票被复用一次(位置 `i`, `i+N`, `i+2N`, ... 都映射到它),冲突恰好有三类。

- 冲突 A(写-写):两个生产者同时想写槽 `i`。

- 冲突 B(读-写):消费者读槽 i 时,生产者正写到半途——撕裂读。
- 冲突 C(覆盖未读):生产者想为新一圈写槽 i ,但上一圈的值尚未被消费——覆盖了活数据。

一切正确性都归结为挡住这三类。下面我们看哪个工具挡住哪个冲突,以及哪些冲突在一个部分解之后幸存——因为幸存者正是逼出下一个机制的东西。

2.1 唯一票号杀死冲突 A——仅此而已

给每次入队一张唯一、单调递增的票号 t (生产者 CAS 抢 `tail_` 计数器;每个值只有一人胜出,败者带着刷新后的 t 重试)。槽位是 $t \& (N-1)$ 。这立刻消除了冲突 A:两个生产者绝不会持有同一张票,因而同一圈里绝不会瞄准同一个槽。

但票 t 和 $t+N$ 相隔一圈映射到同一个槽。票的唯一性对圈数只字未提,所以冲突 B 和 C 幸存。更深的真相:票告诉你谁可以写,但一个被复用的物理槽仍然不知道它此刻处在第几圈——是空着等写,还是满着等读?全局的 `head_/tail_` 计数器无法为单个槽回答这个;它们追踪的是分配,不是每槽的就绪状态。这正是第 1 节那个谎言,如今具体化了。

3. 每槽 token:被逼出来的,不是发明出来的

冲突 B 和 C 都归到一个缺失的事实:对这个物理槽,此刻,是写者的回合还是读者的回合,且是第几圈?全局计数器供不出它。所以这个信息必须住在槽自己身上。我们需要每槽一个标记,回答两个问题:(C)上一圈的值被读走了吗,我可以覆盖吗?(B)这一圈的写完成了吗,我可以读吗?

精妙在于最小编码。我们不需要两个字段(“圈数”加“相位”)。一个单调值——称作槽的 token——就同时编码了二者,因为这个 token 被设为下一张有权操作此槽的全局票号。初始化 `slots_[i].token = i`。于是:

- 持票 t 的生产者,仅当 $\text{token} == t$ 时可写槽 $i = t \& (N-1)$ 。写完后,它通过设 $\text{token} = t + 1$ 发布。
- 持票 h 的消费者,仅当 $\text{token} == h + 1$ 时可读槽 $i = h \& (N-1)$ 。读完后,它通过设 $\text{token} = h + N$ 把槽释放给下一圈。

使这一切成立的那个唯一 trick:一个值同时充当读 guard 与写 guard,二者相差 1。 $\text{token} == t$ 表示“写者 t 的回合”; $\text{token} == t+1$ 表示“读者 t 的回合”; $\text{token} == t+N$ 表示“下一圈写者的回合”。token 单调推进——写 $+1$,读 $+N-1$,故一次完整的“写后读”循环恰好 $+N$,落在下一张映射到此槽的票上。token 是每个槽私有的一个微型单调时钟;生产者与消费者查它,以得知此刻是不是自己的回合。

为什么这两个数必须恰是 $+1$ 和

$+N$ 、而非别的,值得驻足,因为这正是设计最容易设错的地方。写侧的发布,必须把 token

移动“能把它从写者回合翻到读者回合”的最小量: $+1$,得到 $t+1$,而这恰是持票 t

的读者所等待的值。读侧的释放,必须把 token

移到下一张映射到同一槽的票——而共享一个槽的票恰好相隔 N (t 与 $t+N$ 掩码后是同一下标)。所以读者设 $t+N$,即在写者的 $+1$ 之上再加 $N-1$ 。这两个增量是被各自的含义逼定的,不是选出来的: $+1$

是“翻给读者的最小一步”, $+N$

是“交接给下一圈的写者”。任一个设错,槽要么永远不可读、要么永远不可再写。

初始值是同一堂课的缩影,而且值得作为推导真正撞上的一堵墙记录下来。本能的选择是把每个槽的 token 初始化为 0(或干脆零填充)。这立刻就崩:票 1 的生产者要写槽 1、等 $\text{token} == 1$,但零初始化的槽 1 读出

0,于是除槽 0 外每个槽的第一个写者都永远卡住。token 必须被预置为“第一张映射到此处的票”,也就是下标本身: `slots[i].token = i`。唯有如此,槽 i 第一圈的写者(票 i)才会发现 `token == i` 并前进。零初始化的本能是一个会真的走进再去再走出来的陷阱;正确的种子 i 由与 +1、+N 相同的规则决定——token 永远命名下一张有权操作此槽的票,而在零时刻那张票就是 i。

为什么这是整个家族的地基。这个 token 机器里没有任何东西提到有几个生产者、几个消费者。它只假设票被唯一分配。SPSC、MPSC、SPMC、MPMC 共享这个完全相同的每槽状态机;它们的差异仅在于全局 `tail_/head_` 是用普通自增(单侧)还是 CAS(多侧)推进。MPMC 不过是两侧都 CAS 的情形。证一次 token 机器,四种队列的槽级正确性就都证了。又因为票是单调整数、不是被复用的指针,该设计天然免疫 ABA——数组版无需 hazard pointer,不像链表队列。

4. 逐步推导生产者路径

现在我们组装 enqueue,而各部件落位的顺序本身就是这节课。

4.1 先认领,再写

第一个本能——load tail、写 data、再 CAS tail——是错的,而且错得值得内化。在 CAS 成功之前,你不知道自己持有哪张票,因而不知道哪个槽是你的;两个都读到 $t = 0$ 的生产者,会在任一 CAS 落定之前都去写 `slot[0]`——冲突 A 重现。所以认领必须在先:先 CAS 抢票,再写那张确认过的票所指的槽。

4.2 满判断必须随每次重试重做

认领循环还必须判满,而这个判断必须随重试一起走。在循环前判一次 $t - h < N$ 不够: CAS 失败后重试的生产者会拿到刷新后的 t ,而循环前那个陈旧的 h 此刻可能已表示队列满。满判断应放进 CAS 条件里,使每次尝试都重新评估它。

一个细节在测试中浮现,值得诚实记录。当循环因“看起来满了”而退出时,所用的 h 是循环前 load 的那个。由于 `head_` 只增,那个 h 是真实 head 的下界,故队列其实可能已不再满。基于陈旧的 h 宣告失败过于保守,在持续负载下会丢弃本有空位的元素。修法是:在返回 `false` 之前重新 load 一次 `head_`,只有那时才断定队列真的满了。这与 SPSC 的教训如出一辙:满判断必须与对端计数器最新的可用值比较。

4.3 在槽 token 上, CAS 是错误的工具

认领了票 t 之后,生产者必须等到 `slots_[loc].token == t` 才能写。本能是对 token 用 CAS。这是个陷阱。一次失败的 `compare_exchange` 会把槽的当前值写回它的“期望值”实参——但我们的期望值正是票 t ,而它必须保持不变(我们还要用它寻址槽、并发布 $t+1$)。CAS 会污染 t 。更糟的是,我们想要的语义不是“相等则交换”;而是“等到相等,然后写 data,然后设 $t+1$ ”——一次必须发生在数据写之后、而非与比较原子地一起发生的发布。所以正确的构造是 load-spin 循环:反复 load token 直到它等于 t ,全程不动 t ,然后写,然后 store $t+1$ 。

自旋应当礼貌:在等待体里放一条

`_mm_pause()`,降低功耗、缓解内存序流水线惩罚、并向超线程兄弟让出资源。它不是 `sleep`——不进内核、不被调度出去——只是几个周期的礼貌空转。

4.4 acquire 必须落在循环退出那次 load

最后一个细节,也是最常被漏掉的。load-spin 必须建立一条 happens-before 边,使载荷写(生产者)或载荷读(消费者)相对于守护它的 token 被正确定序。人们很容易只把 acquire 放在循环之前那第一次 token load 上。但若第一次 load 尚未看到目标值,就进了循环,而真正放行线程的那次 load——其值最终满足条件的那次——是循环内部的 load。若那次循环体 load 是 relaxed,随后的载荷访问就可能上浮到它之前,使 guard 失效。因此 acquire 必须也在循环体的 load 上(或等价地,在循环后加一道 acquire fence)。定序绑定的是真正让线程前进的那次 load,而不只是第一次试探。

4.5 组装好的生产者

```

bool enqueue(const T& item) {
    uint64_t h = head_.load(memory_order_relaxed);
    uint64_t t = tail_.load(memory_order_acquire);
    while (t - h < N &&
           !tail_.compare_exchange_weak(t, t + 1,
                                         memory_order_release, memory_order_relaxed)) { }
    if (t - h >= N) { // re-check against freshest head
        h = head_.load(memory_order_relaxed);
        if (t - h >= N) return false; // genuinely full
        while (t - h < N &&
               !tail_.compare_exchange_weak(t, t + 1,
                                             memory_order_release, memory_order_relaxed)) { }
        if (t - h >= N) return false;
    }
    uint64_t loc = t & (N - 1);
    uint64_t tok = slots_[loc].token.load(memory_order_acquire);
    while (tok != t) { // load-spin (NOT cas: cas would clobber t)
        _mm_pause();
        tok = slots_[loc].token.load(memory_order_acquire); // acquire on the releasing load
    }
    slots_[loc].data = item; // write AFTER token==t
    slots_[loc].token.store(t + 1, memory_order_release); // publish: now readable
    return true;
}

```

5. 消费者路径:精确的对偶

消费者是生产者的镜像:把 `head_` 与 `tail_` 的角色对调,token 比较整体移 1。它通过对 `head_` 的 CAS 认领一张读票 `h`;队列空当 `h == t`(读票追上了写票)。然后它 load-spin 直到 `token == h + 1`——写者发布的值——读载荷,再以 `token = h + N` 把槽释放给下一圈。同样两条规则成立:把载荷访问写在 token 正确的那一侧,并把 acquire 放在循环退出那次 load。

```

bool dequeue(T& out) {
    uint64_t t = tail_.load(memory_order_relaxed);
    uint64_t h = head_.load(memory_order_acquire);
    while (h < t &&
           !head_.compare_exchange_weak(h, h + 1,
                                         memory_order_release, memory_order_relaxed)) { }
    if (h >= t) { // re-check against freshest tail
        t = tail_.load(memory_order_relaxed);
        if (h >= t) return false; // genuinely empty
        while (h < t &&
               !head_.compare_exchange_weak(h, h + 1,
                                             memory_order_release, memory_order_relaxed)) { }
        if (h >= t) return false;
    }
    uint64_t loc = h & (N - 1);
    uint64_t tok = slots_[loc].token.load(memory_order_acquire);
    while (tok != h + 1) { // wait for writer's publish
        _mm_pause();
        tok = slots_[loc].token.load(memory_order_acquire);
    }
    out = slots_[loc].data; // read BEFORE releasing
    slots_[loc].token.store(h + N, memory_order_release); // release for next cycle
    return true;
}

```

6. 验证

一个多线程压测在容量 1024 的环上用四个生产者、四个消费者运行队列。每个生产者发出一段互不相交的不同整数,使产出值的全集恰好是区间 $[0, \text{TOTAL})$;每个消费者把它出队的每个值记入一个原子 seen 数组,一旦发现重复立即中止。生产者 join、消费者排空之后,压测校验:无任何值缺失,也无任何值重复。

结果。在一百万条(4 生产者 $\times$ 250,000)的多次重复运行中,压测报告 `produced = consumed = 1,000,000,missing = 0,duplicated = 0`——无丢失、无重复、无撕裂或垃圾值。第 4.2 节那个保守满判断的修正是必需的:若不在宣告满或空之前重新 load 对端计数器,压测在持续负载下会丢条目。这个修正恰是 SPSC 的教训:满/空必须以对端计数器最新的值来判定。

7. 与原始 Vyukov 算法的关系

这里推导出的队列,本质上就是 Vyukov 2010 年的有界 MPMC 队列,只是从冲突分析中得来、而非抄来。值得精确指出何处重合、何处分叉,因为分叉之处恰好解释了验证暴露出的那个褶皱。

完全重合之处。每槽 token 机器完全一致。Vyukov 把每个 cell 的 sequence 初始化为其下标(`seq[i] = i`);生产者以 `seq = pos + 1` 发布;消费者以 `seq = pos + N` 释放。这正是上文推出的 $+1 / +N$ 与种子 i ——由“什么翻转 guard / 什么交接圈次”的论证独立得到,且与原版一字不差。槽级状态机是同一个算法。

二者分叉之处:满与空如何判定。Vyukov 从不读对端计数器。他的生产者读 cell 自己的 sequence,算 `dif = seq - pos`; `dif == 0` 表示“轮到我,CAS 抢位置”; `dif < 0` 表示“满,返回 false”; `dif > 0` 表示“有人抢先,重读 pos”。满判断、回合仲裁、重试,被折叠进对 cell 的一次比较里,生产者从不碰 `dequeue_pos`。而这里推出的版本,改用全局差值 `tail_ - head_ < N` 判满(用 `head_ < tail_` 判空),读了对端计数器。这正是第 4.2

节那个保守满判断褶皱的直接成因:因为对端计数器的读可能陈旧,必须在宣告满或空之前重读它——这一步 Vyukov 的 cell 局部判据从不需要。两条通往同一保证的有效路径:全局计数器判据更直观;cell 局部判据更经济(一次比较、无对端计数器流量、每次操作恰好一个 CAS)。

第二个较小的分叉:Vyukov 用单个 `for(;;)` 在一个 CAS 里完成“判就绪 + 抢票 + 退出”;这里的推导把它拆成两个循环——先 CAS 抢票,再 load-spin 槽 token——逐步推理更易,但结构上多一层。

关于等价性。二者是同一算法的两种表述。等价性可由以下两点导出:(a) 每槽 token 机器的同一性——token 取值、增量、种子全部重合;(b) 两种满/空判据的等价性(二者判的是同一个问题——“对应此位置、此圈次的 cell,是否已就绪可供此操作?”,一个从 cell 局部读、一个从计数器全局读)。可在这两点之上构造一个形式化的线性化证明;我们刻意从略,因为本报告的价值在于那条被逼出来的设计链,而非两种等价形式之间的形式对照。

8. 综述

有界 MPMC 队列的每一个特性,都追溯到一个为并行而做的决定:认领位置与写入位置被解耦,于是 tail 在数据落地之前就推进、因而撒谎。从这个谎言起,其余皆被逼出。全局计数器无法为单个槽背书,于是就绪信息

下沉到每个槽、化作一个 token。一个单调 token 就够,因为它命名了下一张有权操作此槽的票,把圈数与相位连同读 guard、写 guard(相差一步)一并编码。在那个 token 上 CAS 是错误的工具,因为一次失败的 CAS 会污染我们必须保住的那张票,于是一个礼貌的 load-spin 取而代之。而 acquire 必须绑定到真正放行线程的那次 load,而非第一次试探。这一切没有一处是凭空的;每一步都是对上一步遗留损害的唯一修补。这正是这个队列值得被推导、而非被抄写的原因——也是为什么同一个 token 机器一旦在此处被理解,SPSC、MPSC、SPMC 便不过是“两张全局票如何推进”的特例而已。